Linux Basics for Hackers

Getting Started with Networking, Scripting, and Security in Kali

by OccupyTheWeb (No Starch Press, 2019)

Comprehensive Book Summary — with Worked Examples

Chapter-by-chapter overview with tools, techniques, and a practical example for each key point

Prepared as a reading companion — not a substitute for the full text.

About This Summary

This document summarizes all 17 chapters of *Linux Basics for Hackers* by OccupyTheWeb — an introductory guide that teaches Linux fundamentals specifically through the lens of offensive security, built entirely around Kali Linux. Each chapter is practical: command concepts are taught alongside their relevance to reconnaissance, exploitation, stealth, and automation.

Each chapter section below includes a synopsis, key techniques (each followed by a shaded example box with real command syntax), and a box listing the **tools/commands** covered. Tools appearing for the **first time** are flagged in orange.

Book Structure (17 Chapters)

- **Core Linux skills** (Ch. 1–7): filesystem/terminal basics, text manipulation, networking, software management, permissions, process management, environment variables.
- **Scripting & system internals** (Ch. 8–10): bash scripting (your first port scanner), compression/archiving, filesystem and storage device management.
- **Stealth, services, and security** (Ch. 11–13): logging and anti-forensics, abusing Apache/OpenSSH/MySQL, staying anonymous (Tor, proxies, VPNs, encrypted email).
- **Wireless & kernel** (Ch. 14–15): Wi-Fi and Bluetooth reconnaissance, kernel modules.
- **Automation & Python** (Ch. 16–17): job scheduling (cron, rc scripts), and Python scripting to build a TCP scanner and a password cracker.

Chapter 1: Getting Started with the Basics

Introduces Kali Linux's terminal and filesystem hierarchy, then the essential survival commands: finding your location (pwd), identifying your user (whoami), navigating directories, and getting help (man). Covers finding files with locate, whereis, which, find, and grep, and basic file/directory manipulation (create, copy, rename, remove).

Key techniques & examples

• pwd and whoami orient you in the filesystem and confirm your current privilege level.

```
$ pwd
/root
$ whoami
root
```

• man pages are the primary reference for every command's full syntax and options.

```
$ man nmap
```

• find performs powerful, flexible searches (by name, type, size, time) across the live filesystem — essential for locating interesting files on a target.

```
$ find / -name "*.conf" -type f 2>/dev/null
```

• grep filters text output for exactly the lines you need, often chained with other commands.

```
$ cat /etc/passwd | grep bash
```

Tools / commands covered in this chapter

pwd	whoami	cd	man
locate	whereis	which	find
grep	touch	mkdir	cp
mv	rm	rmdir	

★ Newly introduced in this chapter: pwd, whoami, man, locate, whereis, which, find, grep

---

Chapter 2: Text Manipulation

Builds on Chapter 1 with deeper text handling: viewing parts of files with head and tail, numbering lines with nl, filtering with grep, and performing find-and-replace operations with sed. Covers paging through files interactively with more and less.

Key techniques & examples

• head and tail let you inspect just the beginning or end of large files (e.g., logs) without opening the whole thing.

```
$ tail -n 20 /var/log/auth.log
```

• sed performs non-interactive search-and-replace across a file or stream, useful for quickly editing config files during an engagement.

```
$ sed 's/PermitRootLogin no/PermitRootLogin yes/' /etc/ssh/sshd_config
```

• less allows searching and scrolling through large files (like logs) without loading the entire file into memory.

```
$ less /var/log/syslog
(press / then a search term, q to quit)
```

Tools / commands covered in this chapter

head	tail	nl	grep
sed	more	less	

★ Newly introduced in this chapter: head, tail, nl, sed, more, less

---

Chapter 3: Analyzing and Managing Networks

Covers inspecting and manipulating network configuration: viewing interfaces with ifconfig, wireless devices with iwconfig, and changing your IP address, netmask, broadcast address, and MAC address (spoofing) for anonymity or evasion. Covers DNS inspection with dig, changing DNS servers, and mapping hostnames to IPs manually.

Key techniques & examples

• ifconfig/iwconfig display and configure network interfaces, the starting point for any network reconnaissance.

```
$ ifconfig eth0
$ iwconfig wlan0
```

• MAC address spoofing can help evade network-based detection or access control.

```
$ ifconfig eth0 down
$ macchanger -r eth0
$ ifconfig eth0 up
```

• dig queries DNS servers directly, revealing a domain's records for reconnaissance.

```
$ dig hackers-arise.com
```

• Manually editing /etc/hosts maps hostnames to chosen IPs, useful for testing or redirection.

```
$ echo "192.168.1.50 target.local" >> /etc/hosts
```

Tools / commands covered in this chapter

ifconfig	iwconfig	macchanger	dig
/etc/resolv.conf	/etc/hosts	dhclient	

★ Newly introduced in this chapter: ifconfig, iwconfig, macchanger, dig

---

Chapter 4: Adding and Removing Software

Covers Debian/Kali package management with apt: searching, installing, removing, updating, and upgrading software. Covers adding new repositories via sources.list, using GUI-based installers, and installing tools directly from source with git — the common way to obtain cutting-edge hacking tools not yet packaged.

Key techniques & examples

• apt is the primary high-level tool for installing and maintaining software on Debian-based systems like Kali.

```
$ apt search exploit
$ apt install exploitdb
```

• Keeping the system updated ensures access to the latest tool versions and security patches.

```
$ apt update && apt upgrade
```

• Adding custom repositories extends the pool of installable software beyond the defaults.

```
$ echo "deb http://example-repo.com/debian/ stable main" >> /etc/apt/sources.list
```

• git clone is the standard way hackers obtain tools distributed only via GitHub, not apt.

```
$ git clone https://github.com/sqlmapproject/sqlmap.git
```

Tools / commands covered in this chapter

apt / apt-get	sources.list	git	

★ Newly introduced in this chapter: apt/apt-get, git

Chapter 5: Controlling File and Directory Permissions

Explains the user/group/other permission model, granting ownership (chown, chgrp), checking and changing permissions (ls -l, chmod, in both decimal/octal and UGO symbolic notation), and setting secure default permissions with umask. Covers special permissions — SUID, SGID, and the sticky bit — and their role in privilege escalation.

Key techniques & examples

• chmod changes read/write/execute permissions using either octal (chmod 755) or symbolic (chmod u+x) notation.

```
$ chmod 755 script.sh
$ chmod u+x script.sh
```

• umask sets the default permission mask applied to newly created files/directories.

```
$ umask 022
```

• SUID lets a program run with the file owner's privileges (often root) regardless of who executes it — a classic privilege-escalation target.

```
$ chmod u+s /usr/bin/custom_tool
$ find / -perm -4000 -type f 2>/dev/null
```

• chown/chgrp transfer ownership of files/directories to another user or group.

```
$ chown root:root /etc/myconfig
```

Tools / commands covered in this chapter

chmod	chown	chgrp	umask
ls -l	SUID/SGID bits		

★ Newly introduced in this chapter: chmod, chown, chgrp, umask

---

Chapter 6: Process Management

Covers viewing running processes with ps (including filtering by name) and top for live, dynamic monitoring of resource-hungry processes. Covers managing process priority with nice, killing processes, running processes in the background, and moving a background process to the foreground.

Key techniques & examples

• ps and top reveal what's running on a system — critical both for your own tool management and for post-exploitation situational awareness on a target.

```
$ ps aux | grep mysql
$ top
```

• nice adjusts a process's CPU scheduling priority, useful for running intensive scans without starving the system.

```
$ nice -n 19 nmap -sS 192.168.1.0/24
```

• kill sends termination signals to misbehaving or unwanted processes by PID.

```
$ kill -9 4821
```

• Appending & backgrounds a long-running task (like a scanner) so the terminal remains usable.

```
$ nmap -sS 192.168.1.0/24 &
$ jobs
$ fg %1
```

Tools / commands covered in this chapter

ps	top	nice	kill
jobs	bg	fg	

★ Newly introduced in this chapter: ps, top, nice, kill

Chapter 7: Managing User Environment Variables

Covers viewing all environment variables and filtering for specific ones, changing variable values temporarily (for a session) or permanently (via startup files), customizing the shell prompt, and modifying the PATH variable — both the correct way and a common mistake that creates a security risk. Covers creating custom variables.

Key techniques & examples

• Environment variables like PATH control how the shell finds and runs commands.

```
$ env | grep PATH
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
```

• Temporarily exporting a variable changes it only for the current session.

```
$ export HISTSIZE=0
```

• Adding to PATH incorrectly (e.g., putting '.' or a writable directory first) can let an attacker hijack commands via a malicious binary with the same name — a key security pitfall.

```
# Risky: export PATH=.:$PATH
# Safer: export PATH=$PATH:/opt/mytools
```

• Custom user-defined variables can store reusable values for scripts.

```
$ export TARGET="192.168.1.100"
$ ping $TARGET
```

Tools / commands covered in this chapter

env	export	PATH variable	PS1 prompt variable
/etc/environment	~/.bashrc		

★ Newly introduced in this chapter: env, export

Chapter 8: Bash Scripting

A hands-on bash scripting crash course: writing and executing your first script ("Hello, Hackers-Arise!"), setting execute permissions, and adding variables and user input. The centerpiece is building your first hacker script: a simple port scanner, then improving it into a MySQL (port 3306) scanner across an IP range using loops and conditionals.

Key techniques & examples

• Every bash script starts with a shebang line declaring the interpreter.

```
#!/bin/bash
echo "Hello, Hackers-Arise!"
```

• chmod +x is required before running a script directly.

```
$ chmod +x HelloHackersArise.sh
$ ./HelloHackersArise.sh
```

• A simple port scanner combines a for loop, /dev/tcp, and conditionals to probe a range of hosts.

```
for ip in 192.168.1.{1..254}; do
(echo > /dev/tcp/$ip/3306) &>/dev/null \
&& echo "$ip: port 3306 open"
done
```

• User input via read lets a script take a target IP/range interactively rather than hardcoding it.

```
read -p "Enter target IP range prefix: " prefix
```

Tools / commands covered in this chapter

bash	chmod +x	read	for loops

/dev/tcp			

★ Newly introduced in this chapter: bash scripting, /dev/tcp

Chapter 9: Compressing and Archiving

Explains compression concepts, then tarring files together into a single archive, compressing with gzip, bzip2, and the older compress utility, and creating bit-by-bit (forensic) physical copies of entire storage devices with dd — useful for both backups and forensic/exfiltration scenarios.

Key techniques & examples

• tar bundles multiple files/directories into a single archive file, often combined with compression.

```
$ tar -cvzf loot.tar.gz /home/target/Documents
```

• gzip and bzip2 compress files to reduce size for storage or exfiltration; bzip2 typically compresses more but is slower.

```
$ gzip largefile.txt
$ bzip2 largefile2.txt
```

• dd creates exact bit-by-bit copies of a storage device, the standard tool for forensic disk imaging.

```
$ dd if=/dev/sdb of=/root/disk_image.img bs=4M status=progress
```

Tools / commands covered in this chapter

tar	gzip	bzip2	compress
dd			

★ Newly introduced in this chapter: tar, gzip, bzip2, dd

Chapter 10: Filesystem and Storage Device Management

Covers the /dev device directory, how Linux represents storage devices and partitions, character vs. block devices, and listing block devices with lsblk. Covers mounting and unmounting storage manually, and monitoring filesystems (disk usage, checking for errors) with df and fsck.

Key techniques & examples

• lsblk lists all block (storage) devices and their partitions, useful for identifying attached drives/USB media.

```
$ lsblk
NAME SIZE TYPE MOUNTPOINT
sda 250G disk
`-sda1 250G part /
```

• mount attaches a filesystem to a chosen directory (mount point) so it becomes accessible.

```
$ mount /dev/sdb1 /mnt/usb
```

• df reports disk space usage across mounted filesystems; fsck checks/repairs filesystem errors.

```
$ df -h
$ fsck /dev/sdb1
```

Tools / commands covered in this chapter

lsblk	mount	umount	df
fsck			

★ Newly introduced in this chapter: lsblk, mount, umount, df, fsck

---

Chapter 11: The Logging System

Covers the rsyslog logging daemon, its configuration file, and its logging rules (facility.priority destination). Covers automatically rotating/cleaning logs with logrotate, then — explicitly framed for stealth — removing evidence of activity from logs and disabling logging entirely to avoid detection.

Key techniques & examples

• rsyslog's config file defines what gets logged, where, based on facility and severity.

```
$ cat /etc/rsyslog.conf
auth,authpriv.* /var/log/auth.log
```

• logrotate automatically archives and trims old logs to prevent unbounded growth.

```
$ cat /etc/logrotate.conf
```

• Selectively removing log entries (or clearing them entirely) is a classic anti-forensic technique covered for awareness/defensive purposes.

```
$ sed -i '/192.168.1.77/d' /var/log/auth.log
```

• Stopping or disabling the logging service entirely prevents new activity from being recorded at all.

```
$ systemctl stop rsyslog
```

Tools / commands covered in this chapter

rsyslog	logrotate	sed (log editing)	systemctl

★ Newly introduced in this chapter: rsyslog, logrotate, systemctl

---

Chapter 12: Using and Abusing Services

Covers starting/stopping/restarting services generally, then three case studies: standing up an Apache web server and editing its index.html, building a covert video "Raspberry Spy Pi" with OpenSSH and a camera module, and extracting information from MySQL (connecting, browsing tables, examining data) plus a note on PostgreSQL with Metasploit.

Key techniques & examples

• systemctl/service manage any Linux service's running state.

```
$ systemctl start apache2
$ systemctl status apache2
```

• A standup Apache server can host a fake login page or payload for social-engineering exercises.

```
$ nano /var/www/html/index.html
```

• OpenSSH provides encrypted remote access, demonstrated here for building a covert remote camera/spy device.

```
$ systemctl start ssh
$ ssh pi@192.168.1.120
```

• mysql client lets you connect to and enumerate a database server directly, a common post-exploitation data-extraction step.

```
$ mysql -u root -p
mysql> SHOW DATABASES;
mysql> SELECT * FROM users;
```

Tools / commands covered in this chapter

systemctl / service	Apache (apache2)	OpenSSH (sshd)	MySQL client
Metasploit (PostgreSQL note)			

★ Newly introduced in this chapter: Apache, OpenSSH, MySQL client

---

Chapter 13: Becoming Secure and Anonymous

Explains how the internet reveals identity (IP addresses, metadata), then covers Tor (how onion routing works and its security concerns), proxy servers (configuring them, security caveats), VPNs, and encrypted email as layered techniques for anonymity and operational security.

Key techniques & examples

• Tor routes traffic through multiple encrypted relays, hiding the originating IP from the final destination.

```
$ apt install tor
$ service tor start
$ torify curl https://check.torproject.org
```

• Proxy servers can be configured at the OS or application level to route traffic through an intermediary, though they typically offer weaker anonymity than Tor.

```
$ export http_proxy="http://127.0.0.1:8080"
```

• VPNs encrypt and tunnel all traffic through a remote provider, hiding activity from local network observers (e.g., an ISP).

```
$ openvpn --config myvpn.ovpn
```

• Encrypted email (e.g., PGP/GPG) protects message content and, with care, sender identity.

```
$ gpg --encrypt --recipient alice@example.com message.txt
```

Tools / commands covered in this chapter

Tor (torify/service tor)	Proxy configuration	OpenVPN	GPG/PGP

★ Newly introduced in this chapter: Tor, OpenVPN, GPG

Chapter 14: Understanding and Inspecting Wireless Networks

Covers basic wireless commands for inspecting Wi-Fi adapters and networks, then Wi-Fi reconnaissance with the aircrack-ng suite (monitor mode, capturing traffic). Covers how Bluetooth works and scanning/reconnaissance against nearby Bluetooth devices.

Key techniques & examples

• iwconfig/iwlist inspect wireless adapters and list nearby visible networks.

```
$ iwlist wlan0 scan | grep ESSID
```

• airmon-ng puts a wireless adapter into monitor mode, a prerequisite for packet capture and injection attacks.

```
$ airmon-ng start wlan0
```

• airodump-ng captures 802.11 traffic and lists visible access points/clients for targeted reconnaissance.

```
$ airodump-ng wlan0mon
```

• hcitool/bluetoothctl discover and inspect nearby Bluetooth devices.

```
$ hcitool scan
```

Tools / commands covered in this chapter

iwconfig	iwlist	aircrack-ng suite (airmon-ng, airodump-ng)	hcitool
bluetoothctl			

★ Newly introduced in this chapter: airmon-ng, airodump-ng, hcitool, bluetoothctl

---

Chapter 15: Managing the Linux Kernel and Loadable Kernel Modules

Explains what a kernel module is, checking the running kernel version, tuning kernel parameters at runtime with sysctl, and managing loadable kernel modules: finding module info with modinfo, and adding/removing modules with modprobe, insmod, and rmmod — relevant both to drivers and to kernel-level rootkits.

Key techniques & examples

• uname reports the running kernel version, important for matching exploits to a target.

```
$ uname -r
5.4.0-kali3-amd64
```

• sysctl reads and modifies kernel parameters at runtime, such as enabling IP forwarding.

```
$ sysctl -w net.ipv4.ip_forward=1
```

• modinfo reveals details about a kernel module before loading it.

```
$ modinfo usbcore
```

• insmod/modprobe load a kernel module into the running kernel — the same mechanism abused by kernel-level rootkits.

```
$ insmod mymodule.ko
$ modprobe -r mymodule
```

Tools / commands covered in this chapter

`uname`	`sysctl`	`modinfo`	`modprobe`
`insmod`	`rmmod`		

★ Newly introduced in this chapter: uname, sysctl, modinfo, modprobe, insmod, rmmod

---

Chapter 16: Automating Tasks with Job Scheduling

Covers scheduling recurring tasks with cron/crontab (including scheduling the MySQL scanner built earlier, and crontab shortcut syntax), then using rc scripts and Linux runlevels to run jobs automatically at system startup, including adding services via rc.d or a GUI.

Key techniques & examples

• crontab schedules a command to run automatically on a recurring time pattern.

```
$ crontab -e
0 2 * * * /root/scripts/mysqlscanner.sh
```

• Crontab shortcut syntax (@reboot, @daily, etc.) simplifies common scheduling patterns.

```
@reboot /root/scripts/startup_check.sh
```

• rc scripts / systemd services launch jobs automatically when the system boots, useful for persistence.

```
$ update-rc.d mytool defaults
```

Tools / commands covered in this chapter

`cron / crontab`	`rc.d scripts`	`runlevels`	`update-rc.d`

★ Newly introduced in this chapter: cron/crontab, rc.d/update-rc.d

---

Chapter 17: Python Scripting Basics for Hackers

Covers installing Python modules with pip, then core Python fundamentals (variables, comments, functions, lists, modules, basic OOP). Builds network communication tools: a TCP client and a TCP listener. Covers dictionaries, control statements, and loops, then closes by improving the hacking scripts built earlier and writing a password cracker using exception handling.

Key techniques & examples

• pip installs third-party Python packages/modules not included in the standard library.

```
$ pip install scapy
```

• A simple TCP client connects to a remote host/port — the foundation for custom scanners and exploit delivery.

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("192.168.1.10", 80))
```

• A TCP listener accepts incoming connections, the basis for a simple reverse-shell handler.

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 9999))
server.listen(5)
```

• Exception handling (try/except) lets a password-cracking script gracefully skip failed attempts and continue through a wordlist.

```
try:
attempt_login(password)
except ConnectionError:
continue
```

Tools / commands covered in this chapter

Python 3	pip	socket module	Python control structures / OOP

★ Newly introduced in this chapter: Python, pip, socket module

Master Tool/Command Index (by first appearance)

This book's tools span core Linux administration commands and dedicated hacking utilities. The table below groups them by category and the chapter where each is introduced.

Category	Ch.	Tools / Commands
Filesystem & search	1	`pwd, whoami, man, locate, whereis, which, find, grep`
Text manipulation	2	`head, tail, nl, sed, more, less`
Networking & DNS	3	`ifconfig, iwconfig, macchanger, dig`
Package management	4	`apt/apt-get, sources.list, git`
Permissions	5	`chmod, chown, chgrp, umask, SUID/SGID`
Process management	6	`ps, top, nice, kill, jobs, bg, fg`
Environment variables	7	`env, export, PATH, PS1`
Scripting	8, 17	`bash, /dev/tcp, Python 3, pip, socket module`
Compression & archiving	9	`tar, gzip, bzip2, dd`
Storage management	10	`lsblk, mount, umount, df, fsck`
Logging & anti-forensics	11	`rsyslog, logrotate, systemctl`
Services	12	`Apache, OpenSSH, MySQL client`
Anonymity	13	`Tor, proxy config, OpenVPN, GPG`
Wireless	14	`aircrack-ng suite (airmon-ng, airodump-ng), hcitool, bluetoothctl`
Kernel management	15	`uname, sysctl, modinfo, modprobe, insmod, rmmod`
Job scheduling	16	`cron/crontab, rc.d, update-rc.d`

Final Notes

Linux Basics for Hackers is explicitly an on-ramp: it assumes no prior Linux experience and teaches just enough of each topic to be useful for the beginning of a hacking/pentesting journey, repeatedly pointing toward the author's Hackers-Arise platform for deeper follow-up material. Every chapter frames otherwise generic sysadmin skills (permissions, logging, services, networking) explicitly through an offensive-security lens.

Because the book is from 2019, some specifics (Kali tool versions, particular package names) may have shifted slightly, but the core Linux commands and concepts it teaches remain foundational and current. The book's final chapters (bash and Python scripting) are intended as a bridge toward writing custom tools rather than relying solely on prebuilt ones.